

Lab Worksheet Report

CNN Training on CIFAR-10 Dataset

Author: Nisarg Upadhyay
Roll Number: B23CS1075
Date: January 31, 2026

Links

Resource	Link
Google Colab	https://colab.research.google.com/drive/1ctEjgG8BoI21D-kZUOJURqR069ptK0zy?usp=sharing
GitHub Pages	https://nisargupadhyaiitj.github.io/MLOps-NisargUpadhyay-B23CS1075/
GitHub Repository	https://github.com/NisargUpadhyayIITJ/MLOps-NisargUpadhyay-B23CS1075
GitHub Branch	https://github.com/NisargUpadhyayIITJ/MLOps-NisargUpadhyay-B23CS1075/tree/Assignment_2
W&B Project Dashboard	https://wandb.ai/b23cs1075-indian-institute-of-technology-/MLOps-Assignment2-CIFAR10?nw=nwuserb23cs1075
W&B Run	https://wandb.ai/b23cs1075-indian-institute-of-technology-/MLOps-Assignment2-CIFAR10/runs/3whvrhq4

1. Experiment Overview

This experiment involves training a SimpleCNN model on the CIFAR-10 dataset for image classification. The notebook implements various MLOps best practices including:

- Custom CIFAR-10 DataLoader implementation
- Model complexity analysis (FLOPs counting with ptflops)
- Gradient flow visualization
- Weight update tracking

- Full Weights & Biases (W&B) integration for experiment tracking
-

2. Environment Setup

- **Device:** CUDA (Tesla T4 GPU)
 - **Framework:** PyTorch
 - **Experiment Tracking:** Weights & Biases (wandb v0.24.0)
 - **FLOPs Calculation:** ptflops library
-

3. Dataset Details

CIFAR-10 Dataset

- **Dataset Size:** 60,000 images (32×32 RGB)
- **Number of Classes:** 10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)

Data Split

Split	Samples
Training	40,000
Validation	10,000
Test	10,000

Data Augmentation (Training)

- **RandomCrop:** 32×32 with padding=4
- **RandomHorizontalFlip:** p=0.5
- **ColorJitter:** brightness=0.2, contrast=0.2
- **Normalization:** mean=[0.4914, 0.4822, 0.4465], std=[0.2470, 0.2435, 0.2616]

Test/Validation Transforms

- **ToTensor** conversion
- **Normalization:** mean=[0.4914, 0.4822, 0.4465], std=[0.2470, 0.2435, 0.2616]

DataLoader Configuration

- **Batch Size:** 128
 - **Training Batches:** 313
 - **Validation Batches:** 79
 - **Test Batches:** 79
 - **num_workers:** 2
 - **pin_memory:** True
-

4. Model Architecture

SimpleCNN Architecture

The model is a simple CNN with ~620K parameters designed for fast training:

```
SimpleCNN(  
    (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True)  
    (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True)  
    (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (bn3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True)  
    (fc1): Linear(in_features=2048, out_features=256, bias=True)  
    (fc2): Linear(in_features=256, out_features=10, bias=True)  
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (dropout): Dropout(p=0.5, inplace=False)  
)
```

Model Summary

Metric	Value
Total Parameters	620,810
Trainable Parameters	620,810
Computational Complexity (MACs)	11.19 MMac

Layer-wise Parameter Distribution

Layer	Parameters	% of Total	MACs	% of MACs
conv1	896	0.144%	917.5 KMac	8.197%
bn1	64	0.010%	65.54 KMac	0.585%
conv2	18.5 K	2.979%	4.73 MMac	42.302%
bn2	128	0.021%	32.77 KMac	0.293%
conv3	73.86 K	11.897%	4.73 MMac	42.228%
bn3	256	0.041%	16.38 KMac	0.146%
fc1	524.54 K	84.493%	524.54 KMac	4.686%
fc2	2.57 K	0.414%	2.57 KMac	0.023%
pool	0	0.000%	57.34 KMac	0.512%
dropout	0	0.000%	0.0 Mac	0.000%

5. Training Configuration

Hyperparameter	Value
Epochs	30
Batch Size	128

Hyperparameter	Value
Learning Rate	0.001
Optimizer	Adam
Weight Decay	1e-4
LR Scheduler	CosineAnnealingLR (T_max=30)
Loss Function	CrossEntropyLoss

6. Training Results

Training Progress (Selected Epochs)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	LR
1	1.6727	37.83%	1.3709	49.57%	0.001000
2	1.3535	50.88%	1.0711	60.63%	0.000997
3	1.2082	56.36%	0.9683	65.24%	0.000989
4	1.1340	59.72%	1.0471	63.23%	0.000976

Note: Training continued for 30 epochs with gradient flow and weight update visualizations generated at epochs 1, 5, and subsequent intervals.

7. Key Features Implemented

7.1 Custom CIFAR-10 DataLoader

- Custom `Dataset` wrapper class with configurable transforms
- Proper train/validation/test split (40K/10K/10K)
- Data augmentation for training set only

7.2 FLOPs Calculation

- Using `ptflops` library for accurate computational complexity analysis
- Per-layer breakdown of parameters and MACs

7.3 Gradient Flow Visualization

- Visualization of gradient magnitudes (max and average) across all layers
- Helps detect vanishing/exploding gradients
- Logged to W&B for experiment tracking

7.4 Weight Update Tracking

- `WeightUpdateTracker` class to monitor learning dynamics
- Tracks mean, max, and std of weight updates between epochs
- Histogram logging to W&B

7.5 W&B Integration

- Full experiment configuration logging
 - Sample image logging
 - Gradient flow plots
 - Weight histograms
 - Training metrics (loss, accuracy, learning rate)
-

8. Observations

8.1 Training Dynamics

- The model shows consistent improvement in both training and validation accuracy over the first few epochs
- Validation accuracy improved from 49.57% (epoch 1) to 65.24% (epoch 3)
- Some fluctuation in validation loss indicates potential overfitting tendencies

8.2 Model Complexity Analysis

- The fully connected layer **fc1** dominates parameter count (84.5%)
- Convolutional layers **conv2** and **conv3** dominate computation (84.5% of MACs combined)
- This suggests potential for optimization through layer redesign

8.3 Gradient Flow

- Gradient visualizations help ensure healthy training without vanishing/exploding gradients
- BatchNorm layers help stabilize gradient flow

9. Conclusion

This experiment successfully demonstrates:

1. Implementation of a custom CNN for CIFAR-10 classification
2. Proper MLOps practices with W&B integration
3. Model complexity analysis using ptflops
4. Comprehensive monitoring through gradient flow and weight tracking

The SimpleCNN model with ~620K parameters achieves reasonable performance on CIFAR-10 while maintaining computational efficiency (11.19 MMacs per forward pass).
